

Desenvolvimento de um Compilador

Prof. Charles Ferreira

1 Instruções

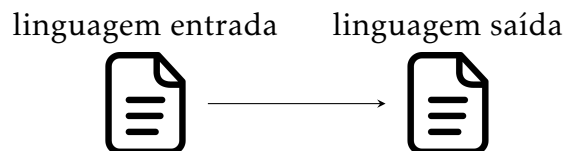
- Este trabalho deverá ser desenvolvido em grupos de **no mínimo** 3 alunos e **no máximo** 5 alunos.
- Entregas: Documentação e projeto.
- A **documentação** deverá ser entregue em formato **pdf** e deverá conter:
 - Título do trabalho.
 - Componentes do grupo.
 - As expressões regulares de todos os tokens.
 - A gramática completa do analisador sintático.
 - Instruções de como executar seu compilador e características da linguagem criada.
 - Exemplos de código na sua linguagem criada e a tradução equivalente.
 - Data de entrega: até **11/05/2026**.
 - **A não entrega da documentação até a data especificada causará redução de 2 pontos na nota final do projeto.**
- O **projeto** deve conter:
 - O código fonte e a aplicação executável (se houver).
 - Data de entrega: **13/05/2026**.
 - **Qualquer atraso na entrega e/ou apresentação resultará em desconto de 2 pontos na nota final do projeto.**
- Durante as aulas dos dias **13/05/2026** e **14/05/2026** cada grupo deverá apresentar a Linguagem de Programação implementada. Todos os integrantes do grupo deverão estar presentes. Caso algum integrante do grupo não possa estar presente então esta pessoa deverá apresentar o trabalho em uma outra data.

A ordem de apresentação será dada por sorteio. Portanto, no dia 13/05/2026 todos devem estar com o projeto pronto.

2 Enunciado

O compilador deve fazer a conversão de um programa desenvolvido na Linguagem definida pelo grupo para uma nova linguagem (fica a critério do professor definir para qual linguagem). **A verificação da corretude do programa** será realizada compilando o arquivo gerado pelo compilador desenvolvido.

Seu compilador deverá receber como entrada um **arquivo** contendo um programa escrito na Linguagem definida pelo grupo e gerar um **outro arquivo** com o código equivalente na linguagem destino, que deverá ser compilada, executada e não deverá conter erros.



OBS: a gramática não pode conter recursividade à esquerda. Caso seja necessário, elimine a recursividade e efetue a sua fatoração.

Cada grupo deve definir a sua própria gramática e todos os tokens necessários. Os requisitos mínimos são:

- Deve ter, no mínimo, 3 tipos de variáveis
- Deve ter a estrutura de controle if ... else;
- Deve possibilitar ifs encadeados
- Deve ter ao menos duas estruturas de repetição (while, do ... while, for);
- Deve possibilitar laços encadeados
- A parte de expressões envolvendo os operadores matemáticos deve ser realizada de maneira correta, respeitando a precedência.
- As atribuições também devem ser realizadas;
- Os comandos de leitura do teclado (Scanner, scanf, input, etc) e de impressão na tela (print) devem ser disponibilizados.
- O compilador tem que aceitar números decimais.
- O compilador deve ter mensagens de erros.

O professor da disciplina decidirá a linguagem destino de cada grupo sendo que alguns grupos poderão ter a mesma linguagem ou cada grupo terá uma linguagem diferente. A linguagem escolhida pelo professor levará em consideração a complexidade necessária para traduzir da linguagem criada pelo grupo. Portanto, o professor informará a linguagem assim que o grupo fizer a gramática da linguagem criada.

3 Exemplo de um Compilador

A descrição a seguir ilustra um exemplo de uma gramática¹ e sua utilização por um compilador que faz a conversão de um programa desenvolvido em uma linguagem fictícia para uma forma equivalente na linguagem C.

Os termos em **negrito** significam palavras reservadas. Preste atenção aos sinais de pontuação.

```
prog → 'programa' bloco 'fimprog'.
bloco → cmd bloco | cmd
cmd → cmdLeitura | cmdEscrita | cmdExpr | cmdSe | declara
declara → tipo ID.
tipo → 'inteiro' | 'decimal'
cmdLeitura → 'leia' '(' ID ')'.
cmdEscrita → 'escreva' '(' conteudo ')'.
conteudo → TEXTO | ID
cmdSe → 'se' '(' expr op_rel expr ')' '{ bloco }' senao '{ bloco }'
cmdExpr → ID ':=' expr.
cp_rel → '<' | '>' | '<=' | '>=' | '!=' | '=='
expr → expr '+' fator | expr '-' fator | expr '*' fator | expr '/' fator | fator
fator → NUM | ID | '(' expr ')'
TEXTO → ' "' (0-9 | a-z | A-Z | ' ' )+ ' "'
NUM → [0-9]+
ID → [a-z]+
```

OBS: espaços em branco, tabs e enter podem aparecer e devem ser eliminados.

¹Esta gramática não atende a todos os requisitos do projeto mas pode ser usada como ponto de partida

Linguagem criada

```
1 programa
2
3 inteiro a.
4 inteiro b.
5 inteiro c.
6 decimal d.
7 escreva("Programa_Teste").
8 escreva("Digite_A").
9 leia(a).
10 escreva("Digite_B").
11 leia(b).
12
13 se(a<b)
14 {
15     c:= a + b.
16 }senao{
17     c:= a - b.
18 }
19
20 escreva("C_e_igual_a_").
21 escreva(c).
22
23 d := c / (a + b).
24
25 escreva("D_e_igual_a_").
26 escreva(d).
27
28 fimprog.
```

Linguagem destino (c)

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a;
6     int b;
7     int c;
8     double d;
9     printf("Programa_Teste");
10    printf("Digite_A");
11    scanf("%d",&a);
12    printf("Digite_B");
13    scanf("%d",&b);
14
15    if(a<b)
16    {
17        c=a+b;
18    }else
19    {
20        c = a-b;
21    }
22
23    printf("C_e_igual_a_");
24    printf("%d",c);
25
26    d = c / (a + b);
27
28    printf("D_e_igual_a_");
29    printf("%lf", d);
30 }
```

4 Critério de Avaliação

O projeto será avaliado mediante os seguintes critérios:

- Analisador Léxico: 3 pontos.
 - fazer todos os tokens mínimos exigidos: 1 ponto.
 - ter a possibilidade de imprimir a lista de tokens: 1 ponto.
 - não utilizar bibliotecas de expressões regulares: 1 ponto.
- Analisador Sintático: 4 pontos.
 - fazer a análise sintática corretamente de todos os tokens: 2 pontos.
 - geração da árvore de derivação: 1 ponto.
 - apresentar mensagens de erros corretas: 1 ponto.
- Tradução correta: 2 pontos.
- Adicionais: até 2 pontos (dependendo da complexidade do adicional).

A apresentação terá caráter eliminatório, ou seja, se não souberem responder as perguntas do professor o projeto será desconsiderado.

Projetos iguais e/ou plagiados serão zerados e encaminhados para a Comissão Disciplinar.

5 Adicionais

Cada grupo deve implementar funcionalidades adicionais que não foram previstas no escopo do projeto.

Exemplos de funcionalidades:

- Fazer a Análise Semântica:
 - Verificar se uma variável foi declarada.
 - Verificar se os tipos de dados de uma expressão são iguais.
 - Verificar o escopo da variável.
- Fazer uma mini IDE para codificar sua linguagem.
- Criar novas regras na gramática que não foram pedidas (Exemplo: classes, funções, interfaces, herança, entre outros. Claramente, não basta criar uma ou duas coisas só).
- Fazer seu compilador ser capaz de compilar para duas linguagens diferentes. Sendo que todas as funcionalidades da sua linguagem deverão ser atendidas para ambas linguagens. Será avaliado a diferença de complexidade entre a linguagem criada e a linguagem traduzida.
- Fazer um Cliente Servidor, ou seja, um compilador rodar como servidor e fica esperando requisições de outros clientes (computadores) com código para ser compilado e devolve o código na nova linguagem ou o erro do programa.
- Outras ideias (conversar com o professor para validar se a ideia possui um mínimo de desafio que faça valer os pontos adicionais).